



AI/ML INTERNSHIP PRACTICAL ASSESSMENT

TOTAL MARKS: 10 | 5 QUESTIONS | 2 MARKS EACH

Assessment Objective

This assessment is designed to evaluate your practical understanding of Python, data analysis, machine learning, visualization, and AI applications.

Submitted By: Rojesh Adhikari

Date: 10/3/2026

Table of Contents

Question 1: Data Cleaning & Student Analysis	3
I. Display the first 10 records and show the number of rows and columns	3
II. Check missing values and duplicate records, then clean them appropriately.....	4
III. Calculate the average attendance, exam score, assignment score, and course completion.	5
IV. Find students whose attendance is below 60% and create at least 2 useful charts.	5
Question 2: Student Risk Prediction	8
I. Select suitable features such as attendance, exam score, assignment score, LMS logins, and course completion.	8
II. Split the data into training and testing sets.	8
III. Train at least one classification model and evaluate it using accuracy and F1-score.	8
Question 3: Attendance Analytics.....	11
I. Find students with attendance below 60%.....	11
II. Calculate average attendance by class or program.	11
III. Create a chart comparing Attendance vs Exam Score.	12
IV. Create an attendance_status column: Good (80%+), Average (60–79%), Low (below 60%).	12
Question 4: Simple Student Analytics Dashboard	14
Question 5: Short AI Research Task.....	16
References.....	40

Question 1: Data Cleaning & Student Analysis

Libraries used during this assessment

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, f1_score
```

Figure 1: Libraries Used

I. Display the first 10 records and show the number of rows and columns

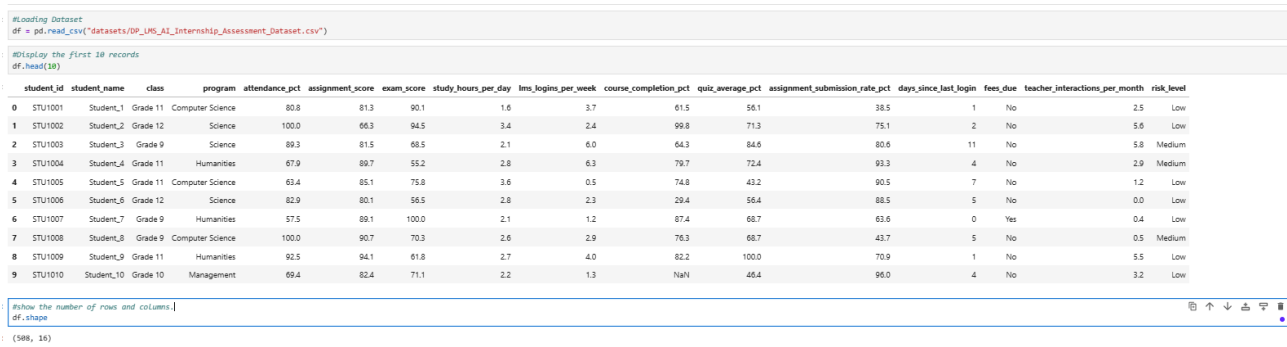


Figure 2: First 10 records and number of rows and columns

II. Check missing values and duplicate records, then clean them appropriately.

```
: #Check missing values and duplicate records
missing_counts = df.isnull().sum()
print("Missing values per column:\n", missing_counts)
duplicate_count = df.duplicated().sum()
print(f"Total duplicate rows: {duplicate_count}")

Missing values per column:
  student_id      0
student_name      0
class            0
program          0
attendance_pct    15
assignment_score  12
exam_score        0
study_hours_per_day  11
lms_logins_per_week  0
course_completion_pct  11
quiz_average_pct  0
assignment_submission_rate_pct  0
days_since_last_login  0
fees_due          0
teacher_interactions_per_month  0
risk_level        0
dtype: int64
Total duplicate rows: 8

: #cleaning them appropriately by placing median in missing columns.
cols = ['attendance_pct', 'assignment_score',
        'study_hours_per_day', 'course_completion_pct']
df[cols] = df[cols].fillna(df[cols].median())
print(df.isnull().sum())

student_id      0
student_name      0
class            0
program          0
attendance_pct    0
assignment_score  0
exam_score        0
study_hours_per_day  0
lms_logins_per_week  0
course_completion_pct  0
quiz_average_pct  0
assignment_submission_rate_pct  0
days_since_last_login  0
fees_due          0
teacher_interactions_per_month  0
risk_level        0
dtype: int64
```

Figure 3: Missing values and duplicate records and datacleaning

III. Calculate the average attendance, exam score, assignment score, and course

```
#Calculate the average attendance, exam score, assignment score, and course completion.  
print("Average Attendance:", df['attendance_pct'].mean())  
print("Average Exam Score:", df['exam_score'].mean())  
print("Average Assignment Score:", df['assignment_score'].mean())  
print("Average Course Completion:", df['course_completion_pct'].mean())
```

```
Average Attendance: 75.86692913385825  
Average Exam Score: 69.1984251968504  
Average Assignment Score: 72.80236220472442  
Average Course Completion: 71.46673228346457
```

completion.

Figure 4: Average scores

IV. Find students whose attendance is below 60% and create at least 2 useful charts.

```
#Students whose attendance is below 60%  
low_attendance = df[df['attendance_pct'] < 60]  
print(low_attendance[['student_id', 'student_name', 'attendance_pct']])
```

	student_id	student_name	attendance_pct
6	STU1007	Student_7	57.5
10	STU1011	Student_11	52.0
14	STU1015	Student_15	53.7
26	STU1027	Student_27	54.5
43	STU1044	Student_44	51.4
44	STU1045	Student_45	59.4
45	STU1046	Student_46	47.5
51	STU1052	Student_52	56.7
52	STU1053	Student_53	52.2
56	STU1057	Student_57	47.1
58	STU1059	Student_59	57.7
73	STU1074	Student_74	59.2
85	STU1086	Student_86	43.8
86	STU1087	Student_87	54.8
96	STU1097	Student_97	53.5
99	STU1100	Student_100	56.1

Figure 5: Attendance below 60%

```
#create at least 2 useful charts.
#1. Students with Attendance Below 60%
plt.figure(figsize=(20, 5))
plt.bar(low_attendance['student_name'],
        low_attendance['attendance_pct'])
plt.xlabel('Student')
plt.ylabel('Attendance (%)')
plt.title('Students with Attendance Below 60%')
plt.xticks(rotation=45)
plt.show()
```

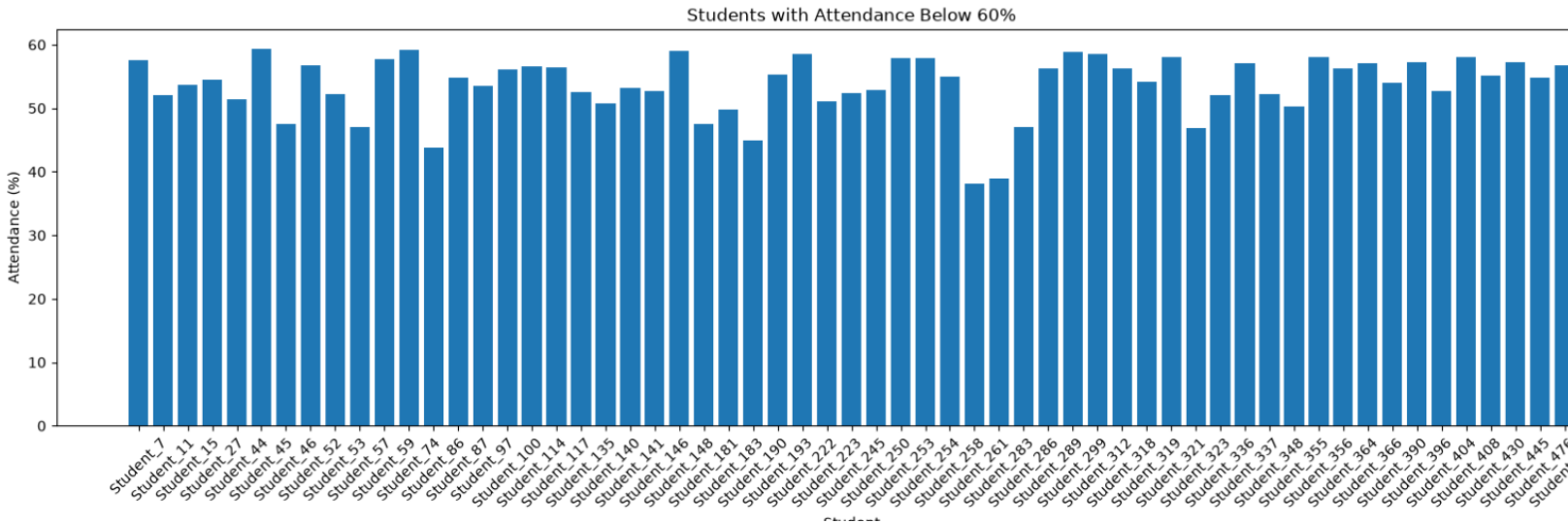


Figure 6: Student with Attendance below 60%

```
#2. Attendance vs Exam Score
plt.scatter(df['attendance_pct'], df['exam_score'])
plt.xlabel('Attendance (%)')
plt.ylabel('Exam Score')
plt.title('Attendance vs Exam Score')
plt.show()
```

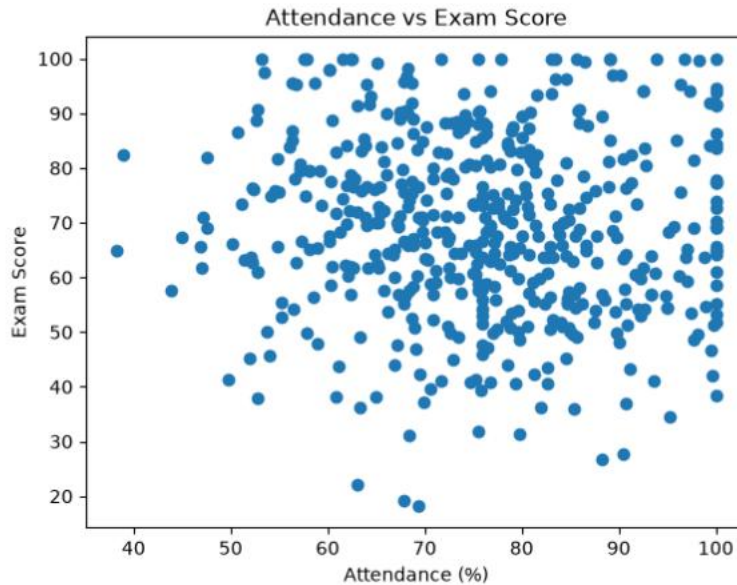


Figure 7: Attendance Vs Exam Score

Two useful insights that I discovered from the datasets are:

- Attendance is a real problem for a sizeable minority: about one in eight students is below 60% attendance, and they are spread fairly evenly across programs (13 to 16 students each) and classes (12 to 17 each), so the intervention needs to be school-wide.
- Attendance alone does not explain results: low-attendance students do not score lower on exams (their average is 72.0 vs 68.7 for the rest), so attendance should be tracked as a separate warning signal rather than used as a proxy for academic performance. Also, the High risk label is extremely rare (4 students), which matters in further modeling.

Question 2: Student Risk Prediction

- I. Select suitable features such as attendance, exam score, assignment score, LMS logins, and course completion.

```
#feature and target selection
X = df[['attendance_pct',
        'exam_score',
        'assignment_score',
        'lms_logins_per_week',
        'course_completion_pct']]

y = df['risk_level']
```

Figure 8: Feature and target selection

- II. Split the data into training and testing sets.

```
#splitting data into training and testing
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

Figure 9: Split data into training and testing

- III. Train at least one classification model and evaluate it using accuracy and F1-score.

```
#training Model with adabooster
weak_learner = DecisionTreeClassifier(
    max_depth=2,
    random_state=42
)
model = AdaBoostClassifier(
    estimator = weak_learner,
    n_estimators = 50,
    random_state = 42,
    learning_rate = 0.1
)
model.fit(X_train, y_train)
```

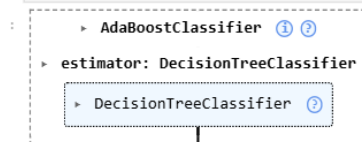


Figure 10: Training model with adabooster

```
#evaluate it using accuracy and F1-score.  
accuracy = accuracy_score(y_test, y_pred)  
f1 = f1_score(y_test, y_pred, average='weighted')  
  
print("Accuracy:", accuracy)  
print("F1-score:", f1)
```

Accuracy: 0.6568627450980392
F1-score: 0.6414054276940439

Figure 11: Accuracy and F1 score

IV. Briefly explain how this prediction could be displayed inside an LMS.

This prediction could be displayed inside an LMS by following ways:

- **At-a-Glance Student Support:** A simple class roster with color-coded badges (Green, Amber, and Red) and risk percentages. Students who may need the most support are automatically shown at the top.
- **Understand Why a Student Is at Risk:** The system shows the top two factors affecting each student's risk level, such as low attendance or poor exam performance. This helps teachers understand the situation and decide how to support the student.
- **Instant Alerts and Quick Actions:** Teachers receive an alert when a student's risk level increases. They can quickly message the student, contact parents, or schedule a mentoring session.
- **Positive Student and Parent Portals:** Instead of using negative labels like "High Risk," students and parents see progress-focused information, including areas for improvement, useful tips, and positive guidance.

- **Teacher-First Governance:** The model updates weekly and works only as a support tool. Teachers have the final say and can review or change a student's risk level based on their own knowledge and experience.
- **Track Student Progress:** Teachers can monitor changes in attendance, academic performance, and course completion over time to see whether students are improving and whether additional support is needed.

Question 3: Attendance Analytics

I. Find students with attendance below 60%.

```
#students with attendance below 60%.
low_attendance = df[df['attendance_pct'] < 60]
print(low_attendance[['student_id', 'student_name', 'attendance_pct']])
```

	student_id	student_name	attendance_pct
6	STU1007	Student_7	57.5
10	STU1011	Student_11	52.0
14	STU1015	Student_15	53.7
26	STU1027	Student_27	54.5
43	STU1044	Student_44	51.4
44	STU1045	Student_45	59.4
45	STU1046	Student_46	47.5
51	STU1052	Student_52	56.7
52	STU1053	Student_53	52.2
56	STU1057	Student_57	47.1
58	STU1059	Student_59	57.7
73	STU1074	Student_74	59.2
85	STU1086	Student_86	43.8
86	STU1087	Student_87	54.8
--	-----	-----	----

Figure 12: Students with attendance below 60%

II. Calculate average attendance by class or program.

```
#average attendance by class.
average_attendance = df.groupby("class", as_index=False)["attendance_pct"].mean()
print(average_attendance)
```

	class	attendance_pct
0	Grade 10	76.248182
1	Grade 11	74.983200
2	Grade 12	75.488667
3	Grade 9	76.885366

Figure 13: Average attendance by class

III. Create a chart comparing Attendance vs Exam Score.

```
#chart comparing Attendance vs Exam Score.
plt.scatter(df["attendance_pct"], df["exam_score"])
plt.xlabel("Attendance")
plt.ylabel("Exam Score")
plt.title("Attendance vs Exam Score")
plt.show()
```

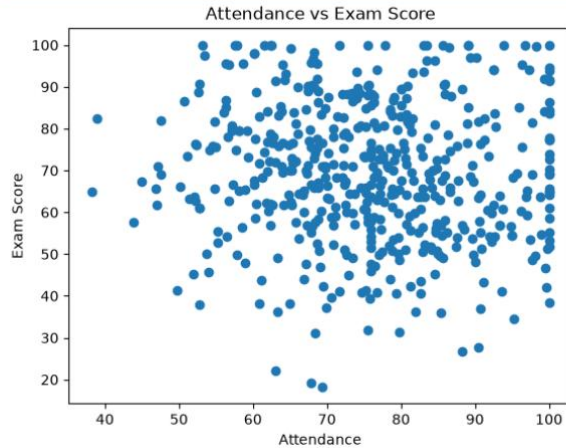


Figure 14: Scatter plot comparing Attendance vs Exam Score

IV. Create an attendance_status column: Good (80%+), Average (60–79%), Low (below 60%).

```
#attendance_status column: Good (80%+), Average (60–79%), Low (below 60%).
df["attendance_status"] = np.select(
    [
        df["attendance_pct"] >= 80,
        df["attendance_pct"] >= 60
    ],
    [
        "Good",
        "Average"
    ],
    default="Low"
)
print(df[["attendance_pct", "attendance_status"]])
```

	attendance_pct	attendance_status
0	80.8	Good
1	100.0	Good
2	89.3	Good
3	67.9	Average
4	63.4	Average
..	...	...
503	92.5	Good
504	80.0	Good
505	60.2	Average
506	52.8	Low
507	95.0	Good

[508 rows x 2 columns]

Figure 15: Creation of attendance status columns.

Analysis:

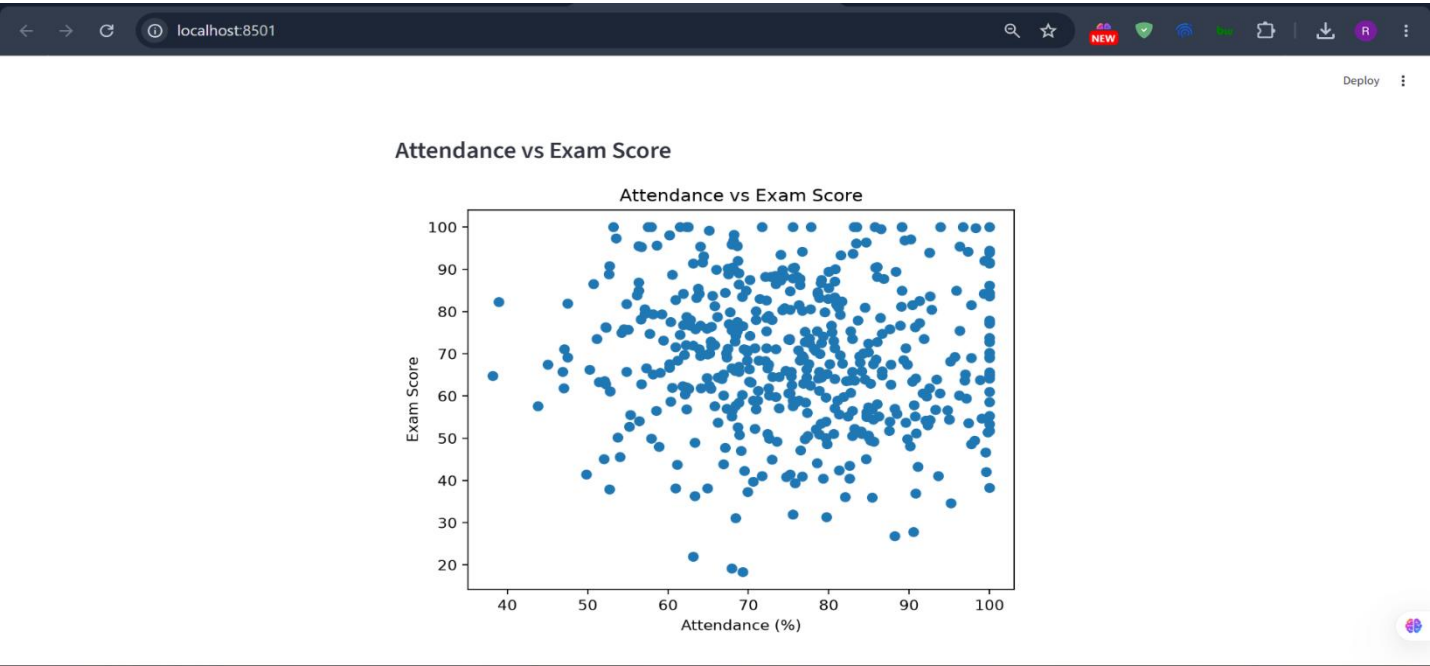
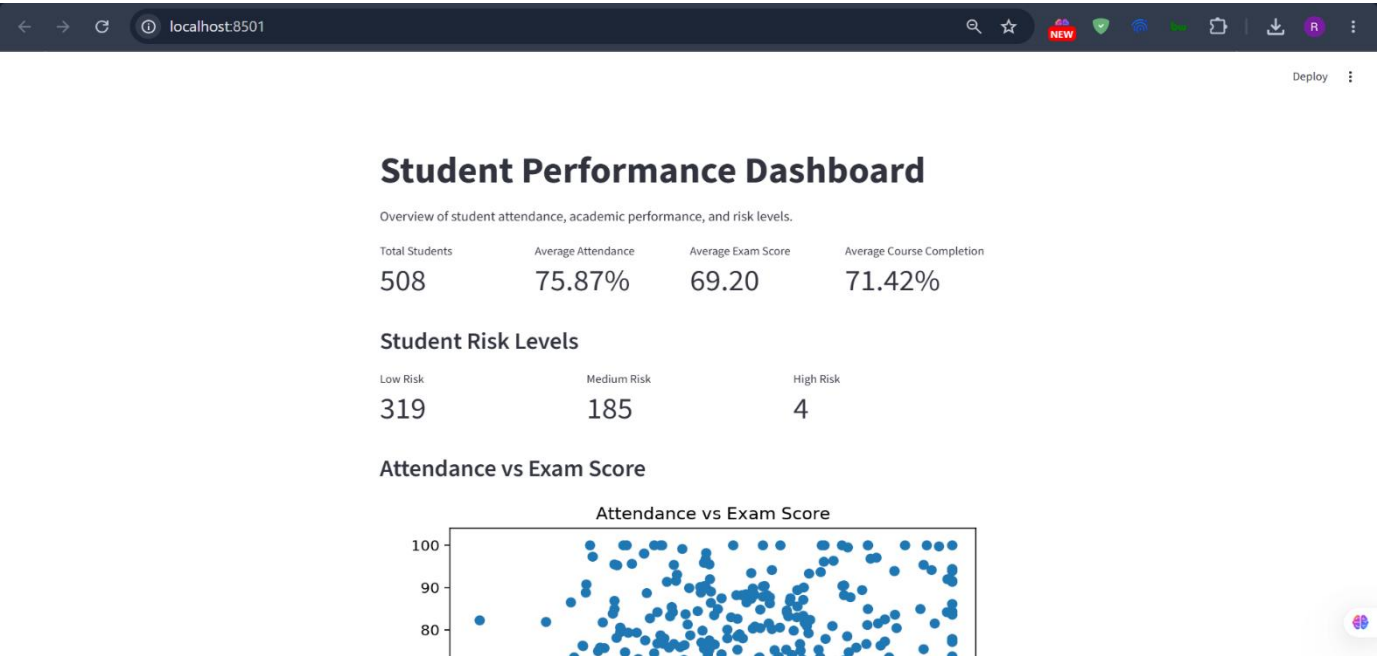
Based on my analysis, the scatter plot shows a weak negative relationship between attendance and exam scores.

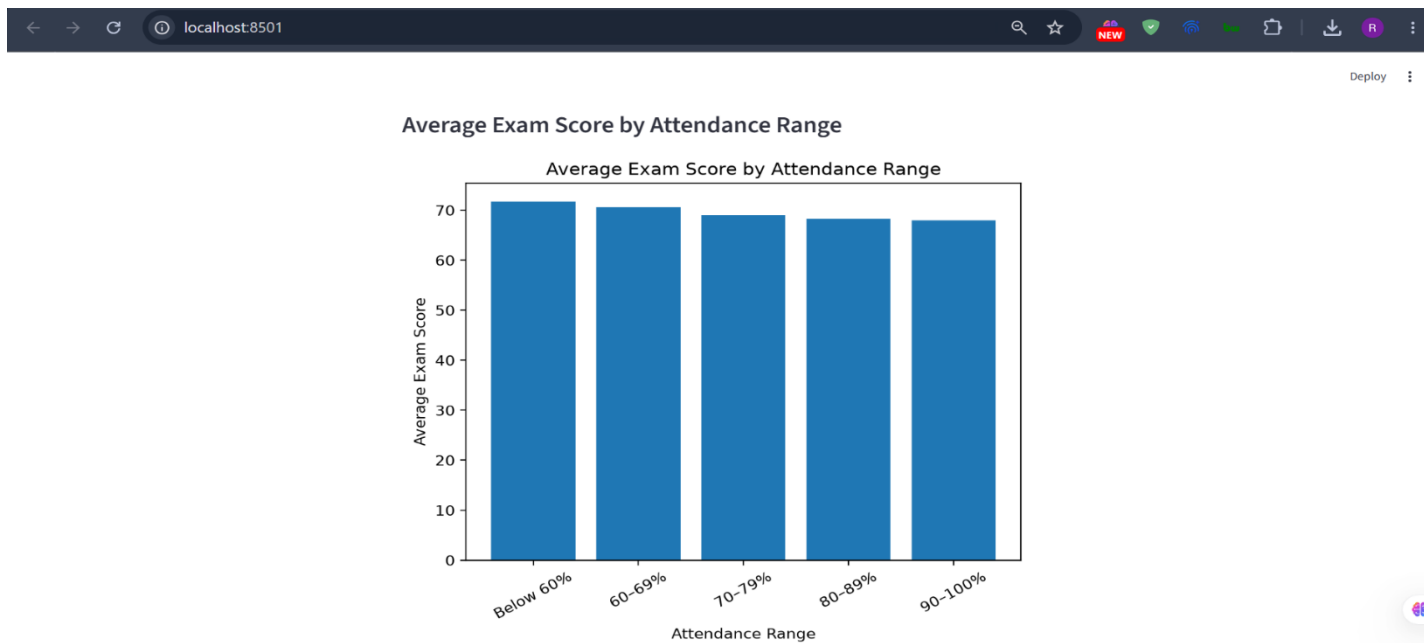
- The points are widely scattered, so attendance does not strongly predict exam performance.
- Students with high attendance can have both high and low exam scores.
- Overall, the trend appears to slope slightly downward, suggesting that higher attendance is associated with a slightly lower exam score in this dataset.
- Therefore, attendance alone does not appear to be a strong factor determining exam performance. Other factors likely contribute significantly.

Question 4: Simple Student Analytics Dashboard

- Libraries used

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import streamlit as st
```





Question 5: Short AI Research Task

Artificial Intelligence can make an LMS more useful by analyzing student activity, identifying learning difficulties, and providing personalized support. For a system such as DP-LMS, AI should be used to assist teachers and students rather than completely replace human decision-making. UNESCO also recommends a human-centered approach to AI in education, with attention to privacy, equity, safety, and meaningful use.

I would choose the following three practical AI features:

1. AI-based Student Performance / Early-Warning System
2. Personalized Learning Resource Recommendation
3. AI Learning Assistant / Automated Feedback

1. AI-Based Student Performance and Early-Warning System

Problem it solves

Teachers may have many students and cannot manually monitor every student's attendance, assignment submission, quiz performance, and LMS activity.

An AI early-warning system can identify students who are at risk of poor academic performance or disengagement. For example, the system could identify a student who:

- Has declining quiz marks
- Has low attendance
- Frequently misses assignments
- Logs into the LMS less often

The teacher could then receive an alert such as:

Example:

Student A may require attention because attendance and assignment completion have declined.

Research has already demonstrated the feasibility of using LMS activity to predict academic achievement. Macfadyen and Dawson analyzed LMS tracking data and developed a predictive model using variables such as discussion activity, messages, and completed assessments. More recent research has also investigated personalized early-warning systems based on students' changing learning states.

Data required

LMS could collect:

Data	Example
Attendance	75%
Assignment marks	62%, 70%, 48%
Quiz scores	80%, 65%, 45%
Assignment submission	7/10 submitted
LMS login frequency	3 times/week
Learning-material activity	Videos viewed, PDFs opened
Previous grades	Previous semester results
Final outcome	Passed/failed or final grade

AI/ML technique

A supervised machine-learning model could be used.

Possible algorithms:

- **Random Forest**

- Logistic Regression
- Gradient Boosting
- Neural Networks

For a first version, I would use Random Forest classification because it works well with a mixture of numerical and categorical features and is relatively straightforward to implement. Scikit-learn provides Random Forest classifiers specifically for classification problems.

The model could classify students into categories such as:

Low risk

Medium risk

High risk

These categories should be treated as indicators for teacher attention, not as definitive judgments about a student's ability.

Implementation in LMS

The AI-Based Student Performance and Early-Warning System could be implemented by connecting a machine-learning model to the existing student data in the LMS/SMS.

Step-by-step implementation process

1. Collect student data

The LMS collects information generated during normal student activity, such as attendance, quiz marks, assignment submissions, exam results, and LMS activity.

2. Store the data

The collected information is stored in the LMS/SMS database for each student. The system can maintain both current and previous academic records.

3. Prepare the data

The system checks the collected data and removes or corrects incomplete or incorrect records. It then calculates useful indicators such as attendance percentage, average quiz score, assignment completion rate, and recent changes in grades.

4. Train the AI model

Historical student data, together with known academic outcomes, is used to train a Random Forest classification model. The model learns patterns associated with different academic outcomes.

5. Analyze current students

Once the model is trained, the LMS provides the latest information about each student to the model. The model analyzes the student's current learning pattern.

6. Generate a risk category

The model produces a risk category such as Low Risk, Medium Risk, or High Risk. For example, a student with declining marks, low attendance, and several missed assignments may receive a higher-risk prediction.

7. Notify the teacher

The prediction is displayed on the teacher's dashboard or generates an alert.

8. Teacher reviews the situation

The teacher checks the student's actual circumstances before taking action. The AI prediction is treated as an indication that a student may need attention, not as a final judgment.

9. Provide support

Based on the teacher's assessment, the student may receive additional learning materials, extra practice, academic guidance, or other appropriate support.

10. Update and improve the model

As more student data and academic outcomes become available, the model can be retrained and evaluated to improve its predictions over time.

The system should be used as a support and early-warning tool rather than an automatic decision-maker. The final decision about a student's academic performance should remain with the teacher. This makes the feature practical for a real LMS/SMS because it uses existing student data, assists teachers in identifying potential problems earlier, and does not require major changes to the existing learning system.

2. Personalized Learning Resource Recommendation

Problem it solves

A normal LMS often provides the same learning materials to every student.

However, students have different:

- Knowledge levels
- Strengths and weaknesses
- Interests
- Learning histories
- Learning speeds

For example, if a student performs poorly on algebra questions, LMS could recommend:

Review: Linear Equations

Watch: Solving Equations

Practice: 10 Basic Equation Questions

Instead of simply showing the entire course content.

Research on LMS recommender systems has explored recommending learning objects based on learner profiles, prior knowledge, performance, and learning behaviour. Recent research also continues to investigate AI-based personalized learning-path recommendation.

Data required

The system could use:

- Student profile
- Previous marks
- Quiz results

- Questions answered incorrectly
- Topics completed
- Time spent on resources
- Videos watched
- Resources downloaded
- Learning objectives
- Difficulty level of resources
- Prerequisites between topics

For example:

Student:

Math score = 55%

Weak topics: Algebra, Quadratic equations

Completed: Basic algebra

Recommended:

1. Algebra revision
2. Linear equation practice
3. Quadratic equation introduction

AI/ML technique

A practical first version could use a recommendation system based on:

- Content-based filtering
- Collaborative filtering

- Student similarity
- Rule-based recommendations

More advanced versions could use:

- Neural networks
- Knowledge graphs
- Embeddings
- Reinforcement learning

Research describes content-based and collaborative filtering as established approaches for educational recommendation systems.

Implementation in DP-LMS

The Personalized Learning Resource Recommendation system could be implemented by connecting a recommendation model to the existing student learning data and the learning resources available in the LMS/SMS.

Step-by-step implementation process

1. Collect student learning data

The LMS collects information generated during normal student activity, such as quiz marks, assignment scores, topics studied, learning resources viewed, time spent on materials, and previous academic performance.

2. Store the data

The collected information is stored in the LMS/SMS database for each student. The system can maintain the student's learning history, including completed topics, previous scores, and resources they have already accessed.

3. Create a student learning profile

The system uses the collected data to create a learning profile for each student. This profile can identify the student's strengths, weak areas, learning progress, and preferred types of learning resources.

4. Identify learning needs

The system analyzes the student's recent performance to identify topics where additional learning support may be required. For example, if a student consistently receives low scores in Algebra, Algebra can be identified as an area that requires further practice.

5. Analyze available learning resources

The LMS maintains information about available resources, such as their subject, topic, difficulty level, and type. Resources may include videos, notes, presentations, articles, examples, and practice quizzes.

6. Match resources with the student

The recommendation model compares the student's learning needs with the available resources. For example, if a student is struggling with Algebra, the system can identify Algebra resources that match the student's current level.

7. Generate personalized recommendations

The system recommends suitable learning materials to the student. For example, it may recommend a basic Algebra video, explanatory notes, and practice questions instead of simply showing all available Algebra resources.

8. Display recommendations in the LMS

The recommended resources are displayed on the student's LMS dashboard or course page. For example:

Recommended for you:

Algebra Basics – Video

Linear Equations – Notes

Algebra Practice Quiz

9. Monitor student interaction

The LMS records how the student interacts with the recommended resources, such as whether the resource was opened, completed, or skipped. It can also monitor whether the student's performance changes after using the recommended material.

10. Update future recommendations

The system uses the student's latest learning activity and performance to update their learning profile and generate new recommendations. This allows recommendations to change as the student's knowledge and learning needs change.

The system should provide personalized recommendations rather than forcing every student to follow the same learning path. This makes the feature practical for a real LMS/SMS because it can use existing student performance and activity data to recommend relevant learning materials. It can also continuously improve recommendations as more information about the student's learning behavior becomes available.

3. AI Learning Assistant and Automated Feedback

Problem it solves

Students often have questions while studying, but teachers may not be available 24/7.

An AI learning assistant could provide immediate assistance with:

- Course questions
- Definitions
- Explanations
- Revision
- Practice questions
- Assignment feedback
- Hints

For example, a student could ask:

Student: *What is photosynthesis?*

The AI could answer using the school's approved course material. It could also provide hints rather than simply giving students the answer.

Research on intelligent tutoring systems has found that automated personalized feedback can improve learning outcomes. One study investigated machine-learning-based personalized hints and explanations in a large-scale intelligent tutoring system.

Data required

The system could use:

- Course notes
- Textbooks or teacher-uploaded materials
- Learning objectives

- FAQs
- Assignment instructions
- Student's current course/topic
- Previous questions
- Quiz results

Importantly, sensitive student information should not be unnecessarily sent to external AI systems. UNESCO specifically highlights privacy and human-centered considerations when implementing generative AI in education.

AI/ML technique

A practical implementation could use:

Large Language Model (LLM) + Retrieval-Augmented Generation (RAG).

Instead of allowing the AI to answer purely from its general knowledge:

Step 1: Student Question

Step 2: Search LMS Course Materials

Step 3: Retrieve Relevant Content

Step 4: Send Context to LLM

Step 3: Generate Answer

This allows the AI to answer based primarily on the school's own learning materials.

Implementation in DP-LMS

The AI Learning Assistant and Automated Feedback feature can be implemented by integrating an LLM-based AI service with the existing LMS. The AI service would communicate with the LMS through an API and use course materials, assignments, quizzes, and student interactions to provide learning assistance and feedback.

How it can be implemented in an LMS

1. Add an AI Assistant to the LMS interface

An "Ask AI" button or chat panel can be added to the student's course page. Students can ask questions about lessons, concepts, assignments, or course materials without leaving the LMS.

2. Connect the AI service through an API

The LMS sends the student's question to an AI service through an API. The AI service processes the request and sends the generated response back to the LMS.

3. Connect the AI with course materials

The AI assistant can be connected to the LMS's existing learning materials, such as:

- Lecture notes
- PDFs
- Course slides
- Videos/transcripts
- Study materials
- Frequently asked questions

A Retrieval-Augmented Generation (RAG) approach can be used so that the AI retrieves relevant course content before generating its response. This helps the assistant provide answers based on the material being taught in that particular course.

4. Integrate AI with assignments

The assignment module can be connected to the AI feedback service. When a student submits a text-based assignment, the LMS sends the submission, assignment instructions, and teacher-defined rubric to the AI for analysis.

5. Generate automated feedback

The AI analyzes the student's response and generates feedback such as:

- What the student did well
- Missing concepts
- Errors or weaknesses
- Suggestions for improvement
- **Possible rubric-based assessment**

For example:

AI Feedback: "Your answer correctly explains the concept, but you should provide an example to support your explanation."

6. Add teacher review and control

A teacher dashboard can display the AI-generated feedback before it is released to the student. The teacher can edit, approve, or reject the AI's suggestions.

This is particularly important when the feedback includes marks or grading recommendations because the AI should assist the teacher rather than make the final academic decision.

7. Provide personalized assistance

The AI can use relevant information from the student's course progress to provide more appropriate assistance. For example, if a student has performed poorly on a particular topic, the assistant can explain that topic, recommend relevant course materials, or generate practice questions.

8. Monitor and improve the feature

The LMS can record interactions such as questions asked, resources accessed, assignment feedback, and student responses. This information can be used to evaluate the usefulness of the AI feature and improve prompts, retrieval, and feedback quality.

This approach makes the feature practical because the existing LMS continues to manage students, courses, assignments, and grades, while the AI service provides an additional layer for question answering, personalized learning assistance, and automated feedback. The teacher remains responsible for final grading and important academic decisions.

Research Question: If you had one month to build one AI feature for DP-LMS, which feature would you choose and why?

If I had one month to build one AI feature for DP-LMS, I would choose an **AI Assignment Feedback Assistant**. This feature would use Artificial Intelligence to analyze students' written assignments and provide suggested feedback, identify strengths and weaknesses, and recommend areas for improvement. Rather than replacing the teacher, the system would act as a support tool, with the teacher reviewing the AI-generated feedback and making marks.

I would choose this feature because providing timely and individual feedback is an important part of learning, but it can be difficult for teachers to provide detailed feedback to every student when they have many assignments to evaluate. Recent research has investigated the use of Large Language Models (LLMs) to evaluate open-ended student responses and generate both quantitative scores and qualitative feedback. For example, a 2024 study comparing LLM-based approaches for mathematics feedback evaluated both scoring accuracy and the quality of generated feedback using teacher judgments and a shared rubric. Another 2024 study investigated GPT-4 for grading student science writing using instructor-provided grades, model answers, and rubrics, finding that the model approximately matched instructor grades in that study. These studies suggest that LLMs have potential as assessment-support tools

For DP-LMS, the feature could use a Large Language Model combined with Natural Language Processing (NLP). The teacher would provide the assignment question and a marking rubric. When a student submits an answer, the system would provide the AI with the question, student's response, rubric, and relevant course material. The AI would then generate feedback such as:

Suggested score: 7/10

Strength: The student correctly explained the main concept.

Weakness: The answer does not provide a practical example.

Improvement: Add an example and explain the concept in more detail.

The teacher would then review, edit, and approve the feedback before it is returned to the student.

Source Code

assessment.ipynb

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, f1_scoreS

#Loading Dataset
df = pd.read_csv("datasets/DP_LMS_AI_Internship_Assessment_Dataset.csv")

#Display the first 10 records
df.head(10)

#show the number of rows and columns.
df.shape

#Check missing values and duplicate records
missing_counts = df.isnull().sum()
print("Missing values per column:\n", missing_counts)
duplicate_count = df.duplicated().sum()
print(f"Total duplicate rows: {duplicate_count}")

#cleaning them appropriately by placing median in missing columns.
cols = ['attendance_pct', 'assignment_score',
        'study_hours_per_day', 'course_completion_pct']
```

```
df[cols] = df[cols].fillna(df[cols].median())  
print(df.isnull().sum())
```

```
#Calculate the average attendance, exam score, assignment score, and course completion.
```

```
print("Average Attendance:", df['attendance_pct'].mean())  
print("Average Exam Score:", df['exam_score'].mean())  
print("Average Assignment Score:", df['assignment_score'].mean())  
print("Average Course Completion:", df['course_completion_pct'].mean())
```

```
#Students whose attendance is below 60%
```

```
low_attendance = df[df['attendance_pct'] < 60]  
print(low_attendance[['student_id', 'student_name', 'attendance_pct']])
```

```
#create at least 2 useful charts.
```

```
#1. Students with Attendance Below 60%
```

```
plt.figure(figsize=(20, 5))  
plt.bar(low_attendance['student_name'],  
        low_attendance['attendance_pct'])  
plt.xlabel('Student')  
plt.ylabel('Attendance (%)')  
plt.title('Students with Attendance Below 60%')  
plt.xticks(rotation=45)  
plt.show()  
  
#2. Attendance vs Exam Score  
plt.scatter(df['attendance_pct'], df['exam_score'])  
plt.xlabel('Attendance (%)')  
plt.ylabel('Exam Score')  
plt.title('Attendance vs Exam Score')  
plt.show()
```

```

#feature and target selection
X = df[['attendance_pct',
        'exam_score',
        'assignment_score',
        'lms_logins_per_week',
        'course_completion_pct']]
y = df['risk_level']

#splitting data into training and testing
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42)


#training Model with adabooster
weak_learner = DecisionTreeClassifier(
    max_depth=2,
    random_state=42
)
model = AdaBoostClassifier(
    estimator = weak_learner,
    n_estimators = 50,
    random_state = 42,
    learning_rate = 0.1
)
model.fit(X_train, y_train)
prediction = model.predict(X_test)
prediction

#evaluate it using accuracy and F1-score.
accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average='weighted')

print("Accuracy:", accuracy)
print("F1-score:", f1)

```

```

#students with attendance below 60%.

low_attendance = df[df['attendance_pct'] < 60]

print(low_attendance[['student_id', 'student_name', 'attendance_pct']])


#average attendance by class.

average_attendance = df.groupby("class", as_index=False)["attendance_pct"].mean()

print(average_attendance)

#chart comparing Attendance vs Exam Score.

plt.scatter(df["attendance_pct"], df["exam_score"])

plt.xlabel("Attendance")

plt.ylabel("Exam Score")

plt.title("Attendance vs Exam Score")

plt.show()

#attendance_status column: Good (80%+), Average (60–79%), Low (below 60%).

df["attendance_status"] = np.select(

    [

        df["attendance_pct"] >= 80,

        df["attendance_pct"] >= 60

    ],

    [

        "Good",

        "Average"

    ],

    default="Low"

)

print(df[["attendance_pct", "attendance_status"]])

#total students, average attendance, average exam score, and average course completion.

print("Total Students:", df.shape[0])

print("Average Attendance:", df["attendance_pct"].mean())

```

```
print("Average Exam Score:", df["exam_score"].mean())  
print("Average Course Completion:", df["course_completion_pct"].mean())
```

```
#the number of Low, Medium, and High-risk students.  
print("Low-risk students:", (df["risk_level"] == "Low").sum())  
print("Medium-risk students:", (df["risk_level"] == "Medium").sum())  
print("High-risk students:", (df["risk_level"] == "High").sum())
```

```
#Attendance vs Exam Score  
plt.scatter(df["attendance_pct"], df["exam_score"])  
plt.xlabel("Attendance")  
plt.ylabel("Exam Score")  
plt.title("Attendance vs Exam Score")  
plt.show()
```

```
#Students by Attendance Status  
df["attendance_status"].value_counts().plot(kind="bar")  
plt.xlabel("Attendance Status")  
plt.ylabel("Number of Students")  
plt.title("Students by Attendance Status")  
plt.show()
```

```
df["risk_level"].value_counts().plot(kind="bar")  
plt.xlabel("Risk Level")  
plt.ylabel("Number of Students")  
plt.title("Students by Risk Level")  
plt.show()
```

dashboard.py

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import streamlit as st

df = pd.read_csv("datasets/DP_LMS_AI_Internship_Assessment_Dataset.csv")

# Page title
st.title("Student Performance Dashboard")
st.write("Overview of student attendance, academic performance, and risk levels.")

# 1. Summary Statistics
total_students = len(df)
average_attendance = df["attendance_pct"].mean()
average_exam_score = df["exam_score"].mean()
average_course_completion = df["course_completion_pct"].mean()

col1, col2, col3, col4 = st.columns(4)
col1.metric("Total Students", total_students)
col2.metric("Average Attendance", f"{average_attendance:.2f}%")
col3.metric("Average Exam Score", f"{average_exam_score:.2f}")
col4.metric("Average Course Completion", f"{average_course_completion:.2f}%")

# 2. Risk Level
st.subheader("Student Risk Levels")
risk_count = df["risk_level"].value_counts()
col1, col2, col3 = st.columns(3)
col1.metric("Low Risk", risk_count.get("Low", 0))
col2.metric("Medium Risk", risk_count.get("Medium", 0))
```

```
col3.metric("High Risk", risk_count.get("High", 0))
```

```
# 3. Chart 1
```

```
# Attendance vs Exam Score
```

```
st.subheader("Attendance vs Exam Score")
```

```
fig, ax = plt.subplots()
```

```
ax.scatter(  
    df["attendance_pct"],  
    df["exam_score"]  
)
```

```
ax.set_xlabel("Attendance (%)")
```

```
ax.set_ylabel("Exam Score")
```

```
ax.set_title("Attendance vs Exam Score")
```

```
st.pyplot(fig)
```

```
# 4. Chart 2
```

```
# Average Exam Score by Attendance Range
```

```
st.subheader("Average Exam Score by Attendance Range")
```

```
df["attendance_range"] = pd.cut(  
    df["attendance_pct"],  
    bins=[0, 60, 70, 80, 90, 100],  
    labels=[  
        "Below 60%",  
        "60–69%",  
        "70–79%",  
        "80–89%",  
        "90–100%"  
    ]  
)
```

```

avg_exam = df.groupby(
    "attendance_range",
    observed=True
)["exam_score"].mean()
fig, ax = plt.subplots()
ax.bar(avg_exam.index, avg_exam.values)
ax.set_xlabel("Attendance Range")
ax.set_ylabel("Average Exam Score")
ax.set_title("Average Exam Score by Attendance Range")
plt.xticks(rotation=30)
st.pyplot(fig)
# 5. Chart 3
# Risk Level Distribution
st.subheader("Student Risk Distribution")
fig, ax = plt.subplots()
risk_count.plot(
    kind="bar",
    ax=ax
)
ax.set_xlabel("Risk Level")
ax.set_ylabel("Number of Students")
ax.set_title("Number of Students by Risk Level")
st.pyplot(fig)

```

References

- Heffernan, N., & Heffernan, C. (2014). The ASSISTments ecosystem: Building a platform that brings scientists and teachers together for minimally invasive research on human learning and teaching. *International Journal of Artificial Intelligence in Education*, 470–497.
- Macfadyen, L. P., & Macfadyen, L. (2010). Mining LMS data to develop an “early warning system” for educators: A proof of concept. *Computers & Education*, 588-599.
- Manouselis, N., Drachsler, H., Verbert, K., & Duval, E. (2011). *Recommender Systems for Learning*. Springer.
- UNESCO. (2023). *Guidance for Generative AI in Education and Research*. UNESCO.